

Lab 4, Act 1 – Answer Key

If we knew the truth, would our trend analysis find it?

NRES 746 – Instructor Answer Key (not for student distribution)

This key follows the same section structure as the student handout. Questions with a single correct numeric or derived answer are answered directly; open-ended items (interpretation, speculation, the plain-English procedure, reflections) are given sample answers and discussion points rather than a fixed rubric. Because Part 4 is a Monte Carlo experiment, every group's numbers will land somewhere in a range, not on an exact decimal – the point of grading is whether their numbers are in the right ballpark and their reasoning about direction (more/less variable, does it agree) is sound, not exact-match precision.

Part 1: The model

1a.

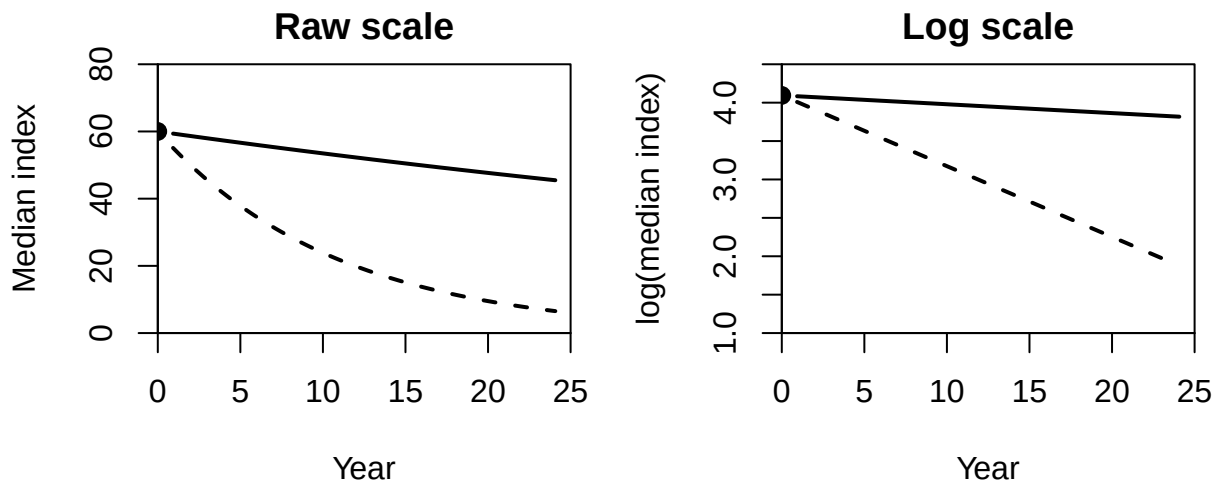
```
lambda <- 0.75^(1/25)
b <- log(lambda)
c(lambda = lambda, b = b)
```

```
##      lambda      b
## 0.98855867 -0.01150728
```

$\lambda \approx 0.9886$, $b \approx -0.01151$. In words: the index falls by about 1.15% every single year, and compounding that for 25 years produces the overall 25% loss.

1b. Only the two **log-scale** curves are straight lines. On the log scale, $\log(\text{median}_t) = \log(I_0) + bt$, which is linear in t with slope b . On the raw scale, $\text{median}_t = I_0\lambda^t$ is an exponential function of t , so both raw-scale curves are curved (concave up for the 25% decline, more sharply curved and flatter-looking for the steeper 90% decline).

```
par(mfrow = c(1, 2), mar = c(4, 4.5, 2, 1))
plot(NA, xlim = c(0, 25), ylim = c(0, 80), xaxs = "i", yaxs = "i",
     xlab = "Year", ylab = "Median index", main = "Raw scale")
curve(60 * 0.75^(x / 25), add = TRUE, lwd = 2)
curve(60 * 0.10^(x / 25), add = TRUE, lwd = 2, lty = 2)
points(0, 60, pch = 16, cex = 1.3)
plot(NA, xlim = c(0, 25), ylim = c(1, 4.5), xaxs = "i", yaxs = "i",
     xlab = "Year", ylab = "log(median index)", main = "Log scale")
curve(log(60) + (log(0.75) / 25) * x, add = TRUE, lwd = 2)
curve(log(60) + (log(0.10) / 25) * x, add = TRUE, lwd = 2, lty = 2)
points(0, log(60), pch = 16, cex = 1.3)
```



1b, continued. A band of $\pm\sigma$ on the log scale is a constant-width pair of parallel lines (since σ doesn't depend on t). Converted back to the raw scale, the band is the median times $e^{\pm 0.2}$ (about $\times 1.2$ and $\times 0.8$), which **shrinks in absolute width as the median shrinks** – the band is a fixed *percentage* of the median, not a fixed number of animals.

```
c(year0_lo = 60 * exp(-0.2), year0_hi = 60 * exp(0.2), year0_width = 60 * (exp(0.2) - exp(-0.2)))

##   year0_lo   year0_hi year0_width
##   49.12385   73.28417   24.16032

c(year25_lo = 45 * exp(-0.2), year25_hi = 45 * exp(0.2), year25_width = 45 * (exp(0.2) - exp(-0.2)))

##   year25_lo   year25_hi year25_width
##   36.84288   54.96312   18.12024
```

The raw-scale band is about 24.2 wide in year 0 (median 60) and only about 18.1 wide in year 25 (median 45, mild decline) – both are exactly 75% of the median, confirming the band scales proportionally rather than staying fixed. A straight-line fit to the raw counts assumes the scatter around the line is the same size everywhere; the assumption being violated is **constant variance (homoscedasticity)** of the residuals. This is exactly the setup behind the optional Act 3's comparison of a raw-scale analyst and a log-scale analyst.

1c. Full raw-scale model:

$$I_t = \underbrace{I_0 \lambda^t}_{\text{deterministic decline}} \cdot \underbrace{e^{\varepsilon_t}}_{\text{noise}} = I_0 e^{bt} e^{\varepsilon_t} = I_0 e^{bt + \varepsilon_t}, \quad \varepsilon_t \sim \text{Normal}(0, \sigma)$$

Taking logs of both sides:

$$\log(I_t) = \log(I_0) + bt + \varepsilon_t$$

The **log-scale** version meets the assumptions of ordinary linear regression: it's linear in t , and the noise ε_t enters additively, independently, normally, and with constant variance σ every year. The **raw-scale** version does not: the noise enters *multiplicatively* rather than additively, the mean function $I_0 e^{bt}$ is nonlinear in t , and (from 1b) its variance is not constant. This is the entire reason the analyst regresses $\log(I_t)$ rather than I_t – it's the version that actually satisfies OLS's assumptions.

Common mistakes to watch for: writing the raw-scale model as $I_0 \lambda^t + \varepsilon_t$ (additive noise on the raw scale) rather than $I_0 \lambda^t \cdot e^{\varepsilon_t}$ (multiplicative) – that's a different, unrelated model, not what 1c describes; forgetting that $\lambda^t = e^{bt}$ and treating λ^t and e^{bt} as if they need separate terms in the log-transformed model (they're two ways of writing the same term, so $\log(\lambda^t) = bt$ exactly).

Part 2: What should the simulation show?

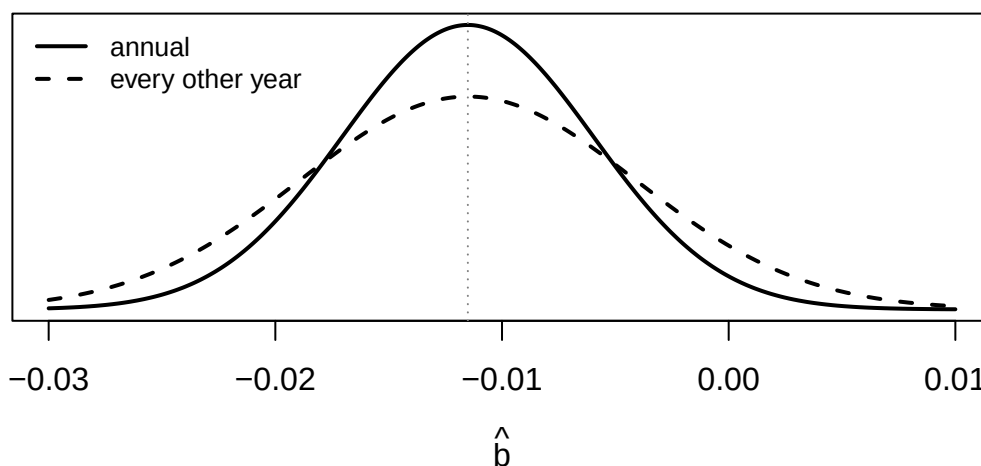
2a. Both designs should be centered at the **same** value, the true b – nothing about *which* years get surveyed changes whether the estimate is centered correctly, only how precisely it pins down b . (Full derivation isn't expected here, but for your own reference: $\hat{b}$ is a weighted average of the log counts with weights that sum to zero and to one when multiplied by t , regardless of which t 's are surveyed, so it's unbiased under any survey schedule.)

The **every-other-year** design should give a **more variable** (wider-spread) set of $\hat{b}$ values, because it uses only 13 data points instead of 25 – fewer independent observations to average over, the same idea as the Week 4 WOD's $\text{Var}(\bar{X}) = \sigma^2/n$ shrinking as n grows. A good answer just needs this direction and the “more data, more precision” reasoning; Part 4C gives the exact formula and factor.

Common misconception to watch for: “the same 25-year span is covered either way, so precision should be about the same” – this misses that a regression slope's precision depends on the *number* of data points used to estimate it, not just the time span they cover. Give credit for correctly identifying annual as more precise even if the reasoning is incomplete; full credit for citing sample size (or, as a bonus insight, that spreading points out in time *also* helps precision, but not enough here to make up for having half as many).

2b. No single correct sketch, but a good one shows both curves centered at the *same* point (near $b \approx -0.0115$), with the annual-survey curve taller and narrower and the every-other-year curve shorter and wider, and both roughly bell-shaped with the same total area underneath (each covers the axis at “1” of probability). For reference, here are the curves the formula in 4C would actually produce:

```
par(mar = c(4, 1, 1, 1))
sd_a <- sigma / sqrt(Stt(tt_annual))
sd_e <- sigma / sqrt(Stt(tt_every2))
xs <- seq(-0.03, 0.01, length.out = 400)
plot(xs, dnorm(xs, b, sd_a), type = "l", lwd = 2, yaxt = "n",
      xlab = expression(hat(b)), ylab = "", ylim = c(0, max(dnorm(xs, b, sd_a))))
lines(xs, dnorm(xs, b, sd_e), lwd = 2, lty = 2)
abline(v = b, col = "grey50", lty = 3)
legend("topleft", c("annual", "every other year"), lty = c(1, 2), lwd = 2, bty = "n", cex = 0.85)
```



2c.

$$\text{SE}(\widehat{\text{coverage}}) = \sqrt{\frac{p(1-p)}{R}} = \sqrt{\frac{0.9 \times 0.1}{R}} = \sqrt{\frac{0.09}{R}}$$

Setting this ≤ 0.01 and solving for R :

```
R_needed <- 0.9 * 0.1 / 0.01^2
R_needed
```

```
## [1] 900
```

$R \geq 900$. The handout's default of 2,000 replicates clears this comfortably (expected MC SE at $R = 2000$ is $\sqrt{0.09/2000} \approx 0.0067$).

Part 3: Plan the experiment

3. No single correct wording, but a complete procedure should include, in some form:

1. Repeat many times (e.g., $R = 2,000$):
 - a. Simulate one set of counts from the **known** data-generating model (using the *true* b and σ) for the survey years being tested.
 - b. Fit $\text{lm}(\log(\text{idx}) \sim t)$ to the simulated counts; extract the slope estimate $\hat{b}$ and its 90% confidence interval.
 - c. Record $\hat{b}$, and whether the interval contains the true b (yes/no).
2. After all R replicates: compute the mean and SD of the R recorded $\hat{b}$ values (bias and variability), and the fraction of replicates whose interval contained the true b (the estimated coverage) – compare that fraction to 90%.

Common omissions to watch for: not being explicit that the *known* (true) b and σ are what generate the fake data – that's the entire point of a “if we knew the truth” check; forgetting to record whether each replicate's interval contains b (needed for the coverage question); describing only a single simulated dataset rather than many repeated ones.

Group check-in

No fixed answer – the point is to reconcile any disagreement in the 2a/2b speculation and agree on one plain-English procedure and target R before splitting into paths.

Part 4A / Part 4B: Simulation results

Both paths run the identical experiment, so expected results are the same regardless of path. Because this is Monte Carlo, every group's numbers will differ slightly; at the handout's $R = 2,000$, expect something close to (one example run shown, with a fixed seed):

```
sim_one <- function(tt, sigma, b, IO = 60, level = 0.9) {
  idx <- IO * exp(b * tt + rnorm(length(tt), 0, sigma))
  m <- lm(log(idx) ~ tt)
  ci <- confint(m, "tt", level = level)
  c(bhat = unname(coef(m)["tt"]), lo = ci[1], hi = ci[2])
}

run_design <- function(tt, sigma, b, R, seed) {
  set.seed(seed)
  res <- t(replicate(R, sim_one(tt, sigma, b)))
  list(bhat = res[, "bhat"], mean_b = mean(res[, "bhat"]), sd_b = sd(res[, "bhat"]),
       coverage = mean(res[, "lo"] <= b & b <= res[, "hi"]))
}

res_a <- run_design(tt_annual, sigma, b, R = 2000, seed = 4746)
res_e <- run_design(tt_every2, sigma, b, R = 2000, seed = 4747)
data.frame(
  design = c("Annual", "Every other year"),
  mean_bhat = c(res_a$mean_b, res_e$mean_b),
  sd_bhat = c(res_a$sd_b, res_e$sd_b),
```

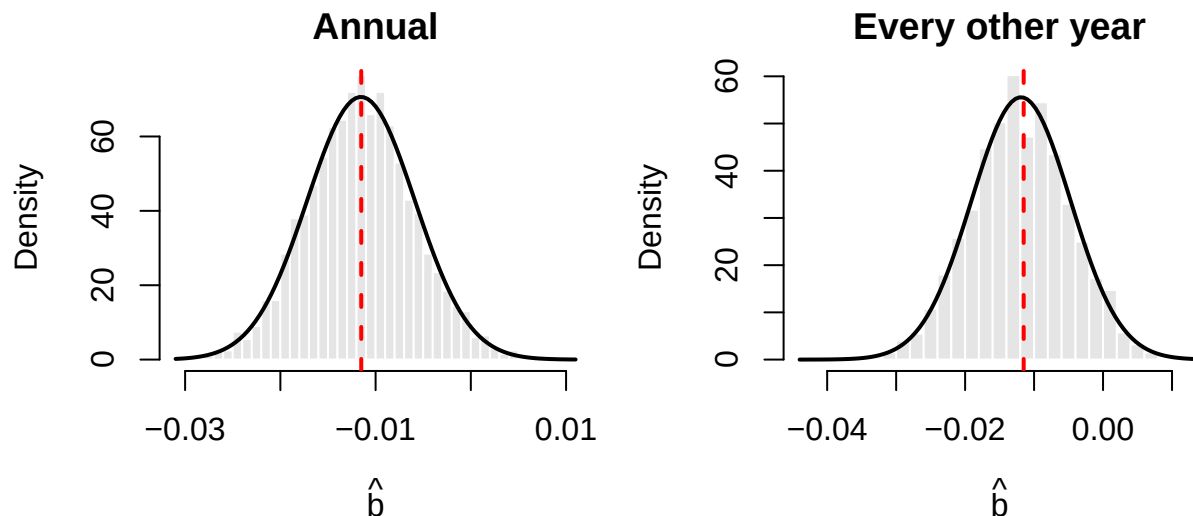
```

coverage = c(res_a$coverage, res_e$coverage)
)

##           design  mean_bhat    sd_bhat coverage
## 1      Annual -0.01153351 0.005647219   0.8980
## 2 Every other year -0.01189395 0.007178580   0.9055

par(mfrow = c(1, 2), mar = c(4, 4, 2, 1))
hist(res_a$bhat, freq = FALSE, breaks = 30, main = "Annual", xlab = expression(hat(b)),
     col = "grey90", border = "white")
curve(dnorm(x, mean(res_a$bhat), sd(res_a$bhat)), add = TRUE, lwd = 2)
abline(v = b, col = "red", lwd = 2, lty = 2)
hist(res_e$bhat, freq = FALSE, breaks = 30, main = "Every other year", xlab = expression(hat(b)),
     col = "grey90", border = "white")
curve(dnorm(x, mean(res_e$bhat), sd(res_e$bhat)), add = TRUE, lwd = 2)
abline(v = b, col = "red", lwd = 2, lty = 2)

```



In both panels, the true b (red dashed line) sits right at the center of the simulated distribution, and a normal curve fit to the simulated mean/SD tracks the histogram well – both things Part 4C asks students to confirm quantitatively.

What to look for, both paths: mean $\hat{b}$ close to the true $b \approx -0.0115$ for both designs (typically within about ± 0.001 at $R = 2,000$); SD of $\hat{b}$ noticeably larger for every-other-year than annual (roughly 30-40% larger); coverage somewhere in the neighborhood of 85-95% for both designs (the exact figure wobbles at $R = 2,000$; see the MC SE from 2c). A coverage estimate far outside that range (below ~80% or above ~98%) usually means a bug, not bad luck – see below.

Common bugs to watch for (both paths, especially AI-generated code):

- Passing survey-design arguments through ... inside `replicate()`: `replicate()` wraps its expression in its own closure, so `function(...)` `Sim(...)` used loosely inside `replicate()` can silently drop or misroute arguments rather than erroring. Worth a spot check if results look implausible (e.g., the index always near zero).
- Calling `confint(m, level = 0.9)` without naming the coefficient ("`tt`" or 2), which can grab the intercept's interval instead of the slope's.
- Rebuilding `tt` incorrectly for the every-other-year design (should be `seq(1, 25, by = 2)`, 13 points) – e.g., accidentally reusing the annual `tt` for both runs, which would make the two designs' results identical.

- Simulating on the raw scale and fitting `lm(idx ~ tt)` instead of `lm(log(idx) ~ tt)` – this silently answers a different question (the raw-scale analyst from the optional Act 3), not this Act’s.

Reflection prompts (4A/4B): no fixed answer. A good AI-assisted reflection names a specific place their own understanding of Parts 1-3 caught something the AI got wrong or wrote ambiguously (e.g., which scale to simulate/fit on, or which coefficient’s CI to use). A good self-coded reflection names a specific sticking point (`confint()` syntax, indexing the coefficient table, vectorizing `replicate()`) and how it got resolved.

Part 4C: Compare with the formula

```
Stt_annual <- Stt(tt_annual); Stt_e2 <- Stt(tt_every2)
sd_annual <- sigma / sqrt(Stt_annual)
sd_e2 <- sigma / sqrt(Stt_e2)
c(annual = sd_annual, every_other_year = sd_e2, ratio = sd_e2 / sd_annual)
```

```
##          annual every_other_year          ratio
##    0.005547002    0.007412493    1.336306210
```

Formula SD: annual ≈ 0.00555 , every-other-year ≈ 0.00741 – the every-other-year design is about **1.34 times** as variable as the annual design (matching the direction of the 2a speculation, and giving it an exact number).

Do they agree? A group’s simulated SD (from `sd(bhat)` over $R = 2,000$ replicates) should typically land within a few percent of the formula value – the relative standard error of a sample SD from n draws is about $1/\sqrt{2(n-1)}$, which is only about 1.6% at $n = 2,000$, so two independent SDs (`sd_a` from 4B and 0.00555 from the formula) agreeing to within roughly $\pm 5\%$ is a good match; agreement within $\pm 15\%$ is still plausibly just Monte Carlo noise. A discrepancy much larger than that (e.g., off by a factor of 1.5 or more, or in the wrong direction) is much more likely to signal a bug than bad luck – a good tell is rerunning the simulation with a different seed: pure Monte Carlo noise moves around between reruns, while a coding bug reproduces the same wrong answer every time.

If you finish early ($\sigma = 0.4$)

Since $\text{SD}(\hat{b}) = \sigma/\sqrt{S_{tt}}$ is *linear* in σ , doubling σ exactly doubles the formula SD:

```
0.4 / sqrt(Stt_annual)
```

```
## [1] 0.011094
```

Formula SD ≈ 0.01109 (versus ≈ 0.00555 at $\sigma = 0.2$). A simulation check ($R=5,000$) gives SD ≈ 0.0111 and mean $\hat{b} \approx -0.0116$ – still centered on the true b . **Coverage should stay close to 90%** even though the noise doubled: `confint()` re-estimates the residual SD from each simulated dataset rather than assuming the true σ , so the 90% interval remains valid regardless of how large σ actually is (a simulation check gives coverage $\approx 90.0\%$). This is worth flagging if a group predicted coverage would drop – doubling the noise makes each individual estimate less precise (wider CI, more spread-out $\hat{b}$), but it doesn’t break the *calibration* of the confidence procedure itself.

Group discussion and revision

No fixed answer – the point is for students to reconcile any real disagreement between their Monte Carlo results and the formula, and to record their own muddiest points.